

Lecture 4: Interpolation and Integration

Computational Bootcamp

John P. Ryan

Summer 2026

University of Wisconsin–Madison

Today's goals

1. Understand the basics of interpolation techniques and how they help solve dynamic models
2. Learn numerical integration (quadrature) for computing expectations

Interpolation

- Remember the optimal capital accumulation problem we solved last week:

$$V(k) = \max_{c, k'} \{ \log(c) + \beta V(k') \}$$

$$c + k' = k^\alpha + (1 - \delta)k$$

- We forced capital to be in a discrete set:

$$D_k = \{k_1, \dots, k_N\}$$

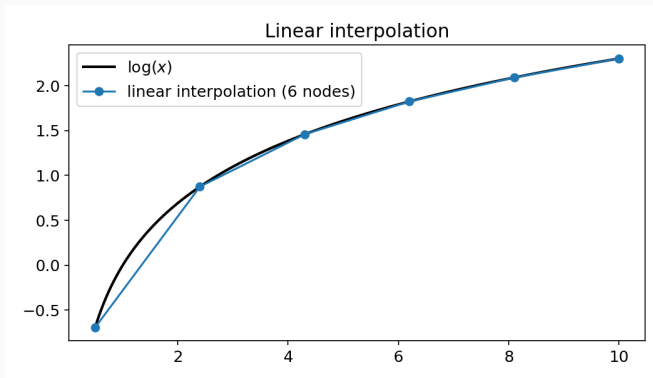
- And solved the value function assuming we could only choose values within this discrete set:

$$V(k) = \max_{k' \in D_k} \{ u(c) + \beta V(k') \}$$

- This is a good way to get started, but:
 1. It forces you to do optimization via grid search
 2. Might need a lot of grid points to get a good approximation
 3. The optimal decision might be between or outside the grid
 4. Don't know what the agent does when between grid points

Interpolation

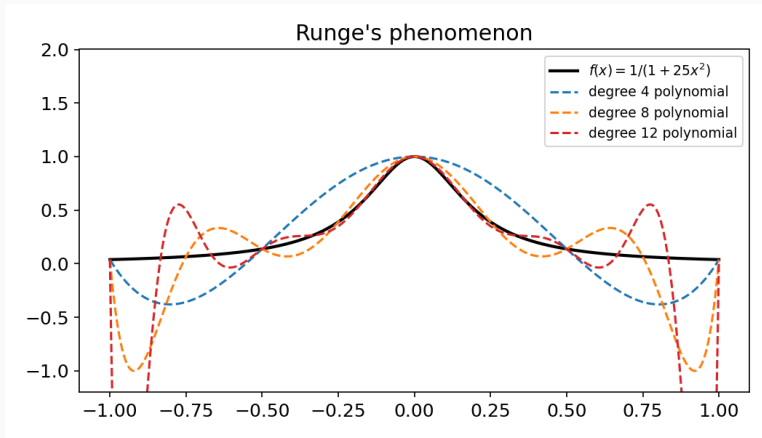
- **Interpolation**: the approximation of a function within a set of known points
- In plain English: connect-the-dots for PhD economists



Types of interpolation

- There are many ways to connect the dots because you can do whatever you want in between them
- Common ways:
 - **Polynomial interpolation**: fit a degree- N polynomial to the data
 - Motivation: any continuous function on compact support can be approximated by polynomials (Stone–Weierstrass)
 - Problem: strange things happen at the tails as N increases (Runge’s phenomenon)
 - **Linear/spline interpolation**: draw lines between points
 - Linear: assume the line is straight
 - Spline: assume the line is a polynomial of low degree (usually cubic)
 - PCHIP: a shape-preserving cubic variant that avoids the overshooting from a cubic spline

Runge's phenomenon



- Higher-degree global polynomials can behave *worse*, especially near endpoints

Linear interpolation

- Discrete domain: $\{x_1, x_2, \dots, x_N\}$; discrete range: $\{f(x_1), \dots, f(x_N)\}$
- Linear interpolation $L(x)$: for $x \in [x_i, x_{i+1}]$,

$$L(x) = f(x_i) + (x - x_i) \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

- Intuition: weighted average of function values based on distance
- Problems:
 1. Derivative can change drastically at grid points
 2. Derivative is constant between grid points

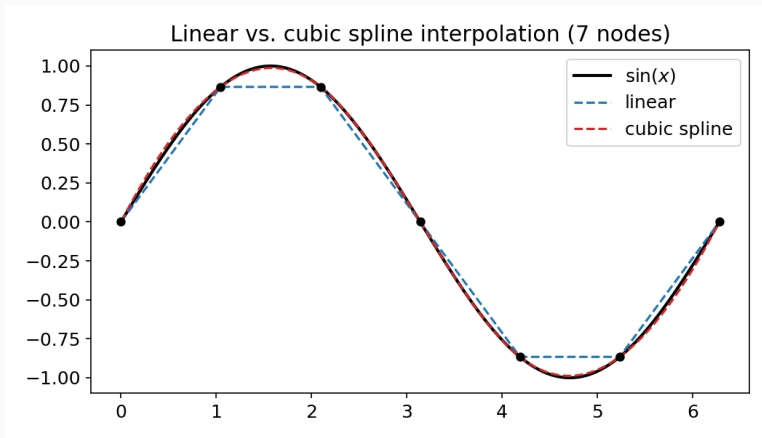
Cubic spline interpolation

- Fit a cubic polynomial between all points: with N points $\{(x_i, f(x_i))\}_{i=1}^N$, for $x \in [x_i, x_{i+1}]$:

$$S_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$$

- N points $\implies N - 1$ intervals and $4(N - 1)$ coefficients, pinned down by:
 - $2(N - 1)$ endpoint conditions: $S_i(x_i) = f(x_i)$ and $S_i(x_{i+1}) = f(x_{i+1})$
 - $2(N - 2)$ continuity conditions on S' and S'' at interior points
 - Two boundary conditions at the ends
- Benefit: smooth approximation with continuous derivatives (nice for optimizers!)
- Cost: can overshoot — added curvature is not always a good thing

Linear vs. cubic in practice



Interpolation in Julia: Interpolations.jl

```
using Interpolations

kgrid = range(0.01, 45.0, length=100)
V = log.(kgrid)                # values on the grid

itp_lin = linear_interpolation(kgrid, V)
itp_cub = cubic_spline_interpolation(kgrid, V)

itp_lin(3.7)                   # evaluate anywhere between grid points
itp_cub(3.7)

# allow (careful!) extrapolation beyond the grid
itp_ext = linear_interpolation(kgrid, V, extrapolation_bc = Line())
```

- Now V is defined *everywhere*: you can hand it to Optim/Roots and choose k' off the grid

PCHIP splines and DataInterpolations.jl

- **PCHIP**: a cubic Hermite spline with slopes chosen to preserve monotonicity — no overshoot, unlike a standard cubic spline

```
using DataInterpolations
```

```
itp_pchip = PCHIPInterpolation(V, kgrid)  # note: (values, points) order!  
itp_pchip(3.7)                           # evaluate anywhere between grid points
```

- `DataInterpolations.jl`: one interface for many interpolants (linear, cubic spline, PCHIP, B-splines, ...) — closer to `scipy.interpolate` than the grid-focused `Interpolations.jl`

Multivariate interpolation

- These methods extend to the multivariate case, e.g. **bilinear** interpolation on a grid $[x_i, x_{i+1}] \times [y_j, y_{j+1}]$:

$$B(x, y) = \frac{1}{(x_{i+1} - x_i)(y_{j+1} - y_j)} \begin{bmatrix} x_{i+1} - x & x - x_i \end{bmatrix} \begin{bmatrix} f(x_i, y_j) & f(x_i, y_{j+1}) \\ f(x_{i+1}, y_j) & f(x_{i+1}, y_{j+1}) \end{bmatrix} \begin{bmatrix} y_{j+1} - y \\ y - y_j \end{bmatrix}$$

- Intuition: interpolate in x , then in y (weighted average of the four corners)
- Interpolations.jl handles this with the same syntax on a 2D grid
- Beware the curse of dimensionality: tensor grids explode as dimensions grow

Integration

Numerical integration: why?

- Expectations are integrals, and economics is full of expectations:

$$\mathbb{E}[V(k', z')] = \int V(k', z') dF(z'), \quad \mathbb{E}[u(c(\varepsilon))] = \int u(c(\varepsilon)) \phi(\varepsilon) d\varepsilon$$

- Distributions often don't give closed forms once wrapped around model objects
- Two broad approaches:
 1. **Monte Carlo**: simulate draws, average
 2. **Quadrature**: deterministic nodes and weights (today)
- For smooth, low-dimensional integrals, quadrature is dramatically more accurate per function evaluation

Quadrature: approximating integrals

- We are interested in approximating an integral (such as an expectation):

$$\int_a^b g(x) dx$$

- Choose a set of n nodes (x_i) and weights (w_i) and calculate:

$$\int_a^b g(x) dx \approx \sum_{i=1}^n w_i g(x_i)$$

- In essence: take a weighted sum of function values instead of evaluating the true integral
- Like Riemann sums from calculus — but with nodes and weights chosen cleverly, n points integrate polynomials up to degree $2n - 1$ *exactly* (Gaussian quadrature)

Quadrature examples

- The right family of nodes/weights depends on the integral:
 - **Gauss-Legendre**: $\int_{-1}^1 g(x) dx$ (bounded intervals, after rescaling)
 - **Gauss-Hermite**: $\int_{-\infty}^{\infty} f(x)e^{-x^2} dx$ (normal shocks!)
- For $X \sim N(\mu, \sigma^2)$: $\mathbb{E}[f(X)] \approx \frac{1}{\sqrt{\pi}} \sum_i w_i f(\mu + \sqrt{2}\sigma x_i)$

```
using FastGaussQuadrature
x, w = gausshermite(9)      # 9 nodes go a long way
Ef = sum(w .* f.(mu .+ sqrt(2)*sigma .* x)) / sqrt(pi)
```

- Sparse grids for higher dimensions: <http://www.sparse-grids.de>

What about correlated normals?

- Suppose the model has two *correlated* normal shocks
- Use quadrature (or simulation) for *independent* standard normals, then transform with the **Cholesky decomposition** $\Sigma = LL'$:

$$\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim N(0, \Sigma) \implies \begin{aligned} X_1 &= L_{11}Z_1 \\ X_2 &= L_{21}Z_1 + L_{22}Z_2 \end{aligned} \quad Z_1, Z_2 \text{ iid } N(0, 1)$$

- Bonus for estimation: the coefficients of L enter *linearly*, so estimate the (unconstrained) entries of L and back out $\Sigma = LL'$ — positive semi-definiteness for free
- In Julia: `cholesky(Sigma).L`

- Interpolation + optimization + quadrature are the three ingredients of serious dynamic programming:
 - Interpolate V off the grid, optimize over continuous k' , integrate over shocks
- Start with linear interpolation and a coarse grid; refine only once things work
- Check quadrature accuracy by doubling the number of nodes
- To the code!